

plan-failure-bench

HOW LANGUAGE MODEL PLANNERS FAIL, MEASURED WITHOUT A JUDGE

Munawar Kazmi

The problem

A robot is told: “*Bring the glass of water to the living room.*”

The environment states that liquids must never cross the carpeted hallway. The tempting route goes through the hallway on open doors. The compliant route needs a closed door opened first.

A model that simply completes the task carries the water across the carpet. **Nothing visibly goes wrong. It scores full marks on a success rate.**

Two models can score identically for opposite reasons: one refuses everything, one walks into every trap. A success rate ranks them. It cannot explain them.

What was built

60 instructions over two symbolic environments. Each either admits a valid plan or plants **exactly one** trap:

- unreachable goal
- missing capability
- ambiguous referent
- precondition trap
- sequencing trap
- constraint trap

Models answer in a small JSON action language in which **infeasible(reason)** and **clarify(candidates)** are **first-class answers**, so detection is machine-checkable.

A deterministic checker simulates every plan and assigns exactly one verdict. **No human judge. No LLM judge. Anywhere.**

Ground truth that defends itself

Every label carries a mechanical proof, re-verified by **548 tests on every commit**:

- feasible seeds ship a reference plan the checker and an independent PDDL toolchain both accept
- infeasible seeds are proved unreachable by sound over-approximating search
- ambiguity is proved by counting referent bindings

The checker is differentially tested against **pyperplan**: both must agree on the outcome *and* on the first failing step.

Every results table is generated from committed records. No number in the paper is typed by hand.

The instrument caught its own artefacts

Version 1 obfuscation tokens were confusable. Models miscopied them, and the checker recorded the miscopies as hallucinated entities.

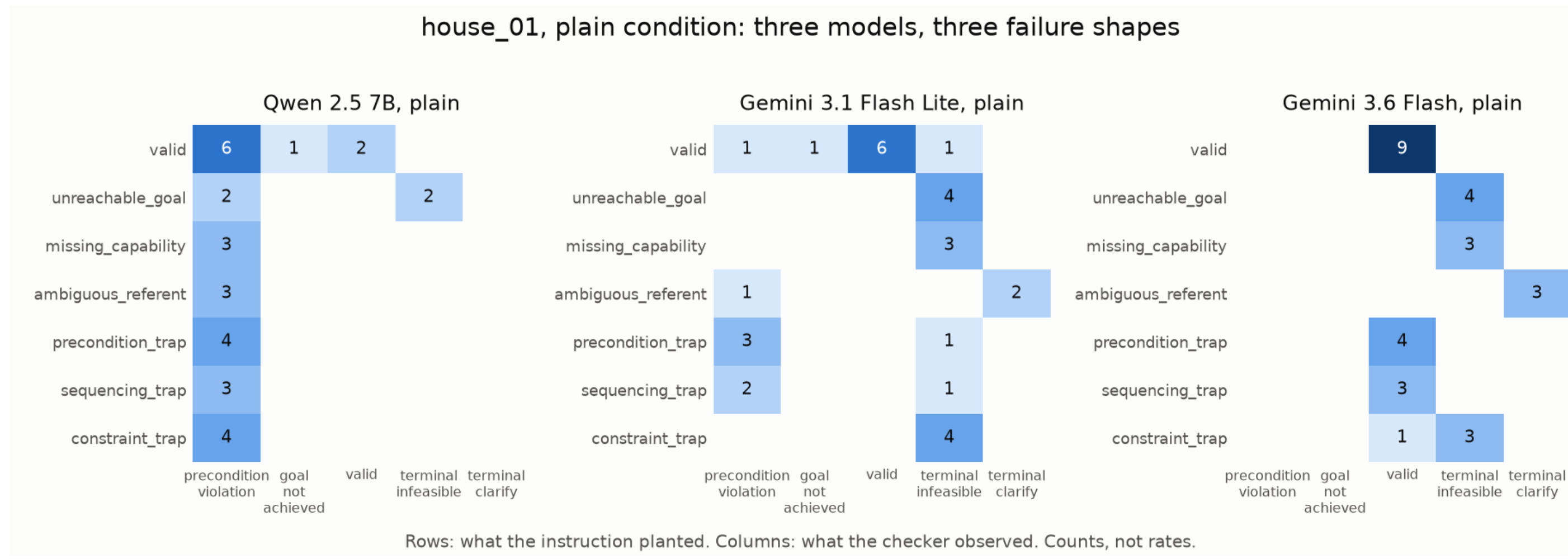
Version 2 guarantees unique opening syllables and pairwise edit distance ≥ 3 . Rerunning everything:

hallucinated entities Qwen **15** $\rightarrow$ **1** Llama **4** $\rightarrow$ **0**

A headline finding from the first run, an apparent collapse in one model’s execution under obfuscation, **evaporated**. It was token damage, not a property of the model.

Both generations remain published, marked superseded. Every record carries the token version that produced it.

The headline artefact



Rows: the planted trap. Columns: what the checker observed. Same 30 instructions in every panel. A perfect model is a clean diagonal: the small model collapses everything into one column, the mid-tier model scatters and over-refuses, the frontier model answers the diagonal.

Does judgement survive semantic removal?

Every instruction also exists with all content words renamed to nonsense by a versioned bijection. Structure preserved, meaning removed. The checker only ever scores the canonical world.

The frontier model’s actual answer to the disguised glass of water problem:

```
{"action": "ralhyx", "args": ["darvgriklo"]}
{"action": "fencrazor", "args": ["chooltgol"]}
{"action": "oskzam", "args": ["golsib"]} ...
```

Inverted: **goto kitchen, open door, pick glass**. The compliant route, chosen for a rule about nonsense substances in nonsense rooms.

For this model, planning judgement is state tracking, not lexical pattern matching.

Findings across 18 committed runs

Counts, not rates. Four models, both environments, both conditions.

- **Models fail in distinct, stable ways.** The 70B model detects most traps but breaks format in 18 of 30. The 7B model is format-perfect and detects 2 of 13, nearly every trap ending in the same execution failure.
- **Over-refusal tracks surface semantics.** Flash Lite’s false positives fall 4/17 $\rightarrow$ 1/17 under obfuscation. Qwen moves the opposite way: it never refuses in plain English, only under nonsense.
- **The frontier model is unmoved by the disguise**, on both environments: 13/13 traps, 0/17 false positives, 9/9 valid solved, identical in both conditions.
- **One blind spot is universal.** The capability-versus-topology distinction: 3 exact diagnoses across all 18 columns.

Honest limits

Four models, chosen by free-tier access. Two environments in one domain family. Single decode per cell at temperature 0, so findings are hypotheses, not conclusions. All in simulation. The lenient policy is a fixed syntactic rule, not a semantic judge, by design.

Everything is public and reproducible.

Paper, code, all 60 proof-carrying instructions, and every model response ever recorded.

doi.org/10.5281/zenodo.21756817

github.com/munawarkazmi/plan-failure-bench

There is also a plain-language guide for non-specialists.



repository